Binary Image Analysis

Binary image analysis

- consists of a set of image analysis operations that are used to produce or process binary images, usually images of 0's and 1's.

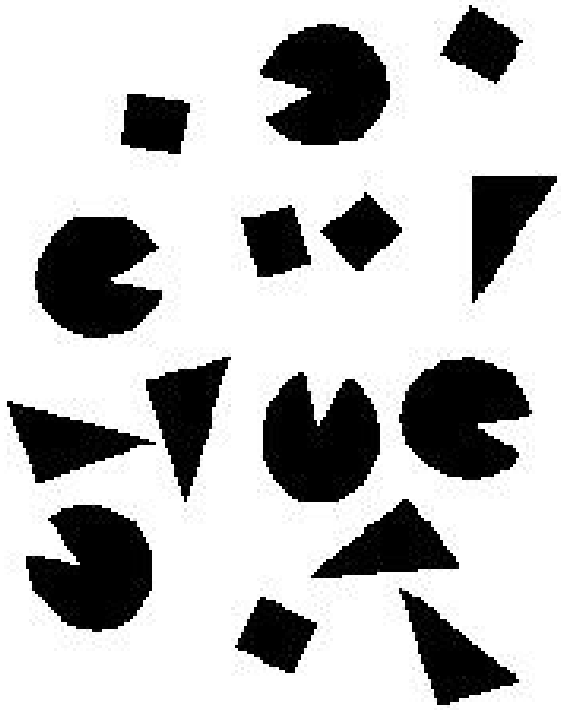
0 represents the background
1 represents the foreground

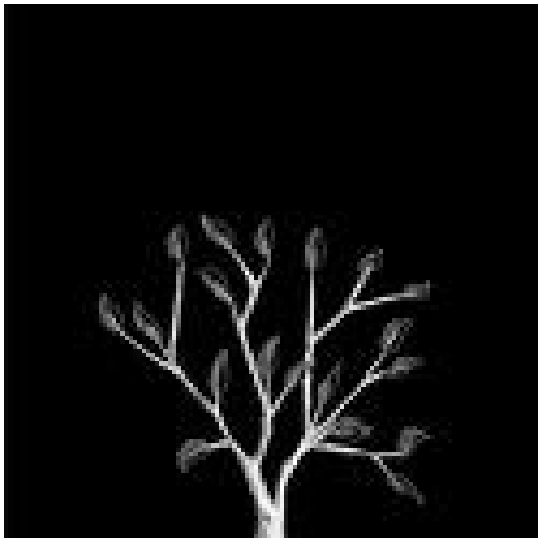
```
0001001000 1000
0001111000 1000
0001001000 1000
```

Binary Image Analysis

is used in a number of practical applications, e.g.

- part inspection
- riveting
- fish counting
- document processing







part of Hypertext Transfer Protocol -- HTTP/1.1
RFC 2616 Fielding, et al.

1 Introduction

1.1 Purpose

The Hypertext Transfer Protocol (HTTP) is an application-level protocol for distributed, collaborative, hypermedia information systems. HTTP has been in use by the World-Wide Web global information initiative since 1990. The first version of HTTP, referred to as HTTP/0.9, was a simple protocol for raw data transfer across the Internet. HTTP/1.0, as defined by RFC 1945 [6], improved the protocol by allowing messages to be in the format of MIME-like messages, containing metainformation about the data transferred and modifiers on the request/response semantics. However, HTTP/1.0 does not sufficiently take into consideration the effects of hierarchical proxies, caching, the need for persistent connections, or virtual hosts. In addition, the proliferation of incompletely-implemented applications calling themselves "HTTP/1.0" has necessitated a protocol version change in order for two communicating applications to determine each other's true capabilities.

This specification defines the protocol referred to as "HTTP/1.1". This protocol includes more stringent requirements than HTTP/1.0 in order to ensure reliable implementation of its features.

Practical information systems require more functionality than simple retrieval, including search, front-end update, and annotation. HTTP allows an open-ended set of methods and headers that indicate the purpose of a request [47]. It builds on the discipline of reference provided by the Uniform Resource Identifier (URI) [3], as a location (URL) [4] or name (URN) [20], for indicating the resource to which a

method is to be applied. Messages are passed in a format similar to that used by Internet mail [9] as defined by the Multipurpose Internet Mail Extensions (MIME) [7].

HTTP is also used as a generic protocol for communication between user agents and proxies/gateways to other Internet systems, including those supported by the SMTP [16], NNTP [13], FTP [18], Gopher [2], and WAIS [10] protocols. In this way, HTTP allows basic hypermedia access to resources available from diverse applications.

1.2 Requirements

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119 [34].

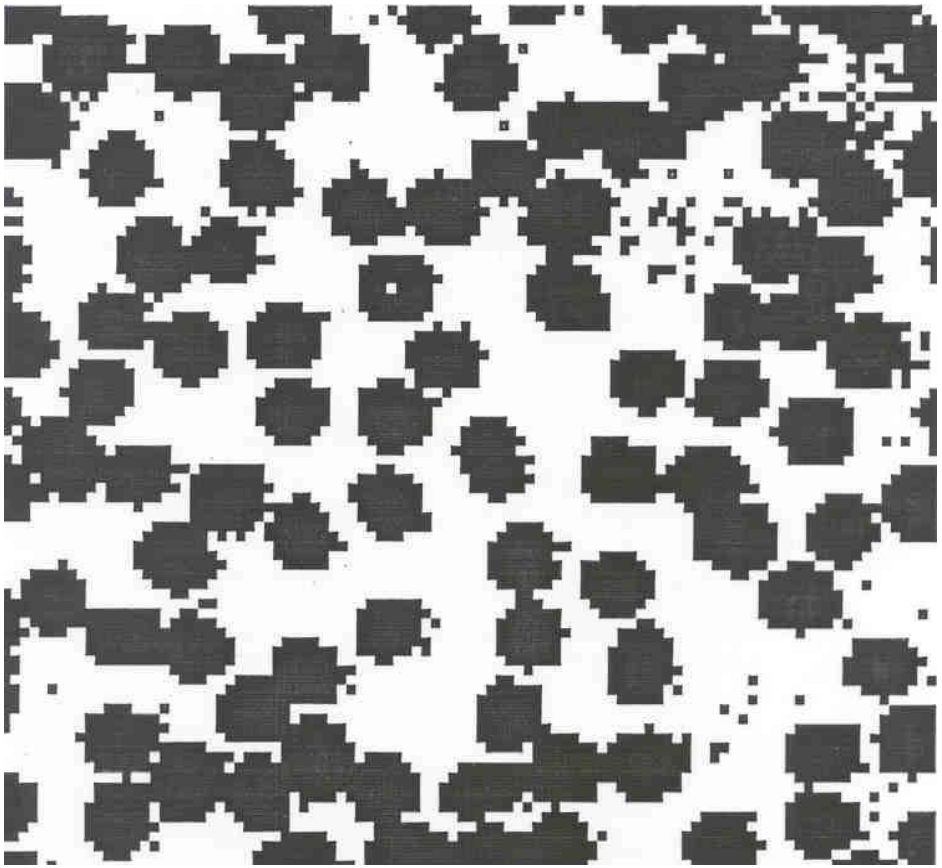
An implementation is not compliant if it fails to satisfy one or more of the MUST or REQUIRED level requirements for the protocols it implements. An implementation that satisfies all the MUST or REQUIRED level and all the SHOULD level requirements for its protocols is said to be "unconditionally compliant"; one that satisfies all the MUST level requirements but not all the SHOULD level requirements for its protocols is said to be "conditionally compliant."

1.3 Terminology

What kinds of operations?

- Separate objects from background and from one another
- Aggregate pixels for each object
- Compute features for each object

Example: red blood cell image



- Many blood cells are separate objects
- Many touch – bad!
- Salt and pepper noise from thresholding
- How useable is this data?

Results of analysis

- 63 separate objects detected
- Single cells have area about 50
- Noise spots
- Gobs of cells

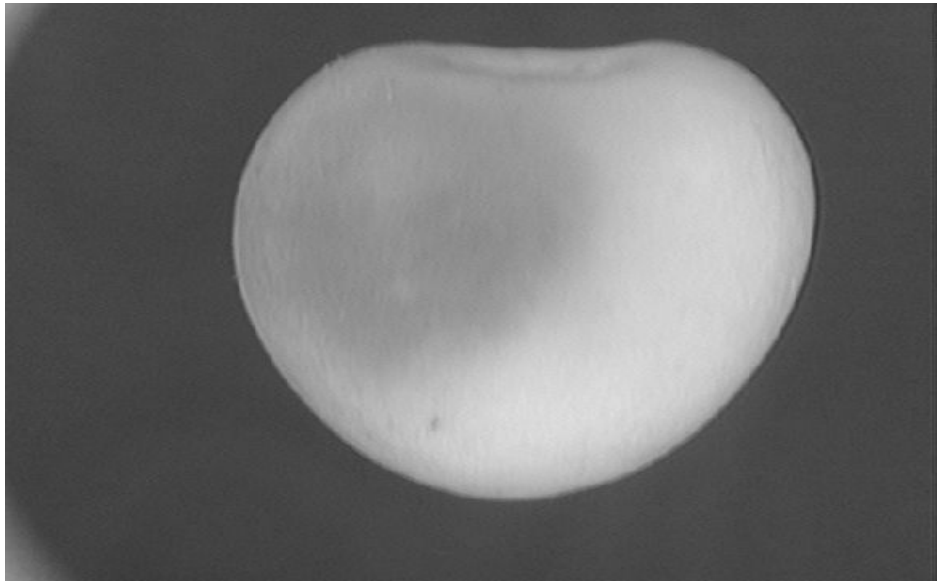
Object	Area	Centroid	Bounding Box	
1	383	(8.8 , 20)	[1 22 1 39]	
2	83	(5.8 , 50)	[1 11 42 55]	
3	11	(1.5 , 57)	[1 2 55 60]	
4	1	(1 , 62)	[1 1 62 62]	
5	1048	(19 , 75)	[1 40 35 100]	gobs
32	45	(43 , 32)	[40 46 28 35]	cell
33	11	(44 , 1e+02)	[41 47 98 100]	
34	52	(45 , 87)	[42 48 83 91]	cell
35	54	(48 , 53)	[44 52 49 57]	cell
60	44	(88 , 78)	[85 90 74 82]	
61	1	(85 , 94)	[85 85 94 94]	
62	8	(90 , 2.5)	[89 90 1 4]	
63	1	(90 , 6)	[90 90 6 6]	

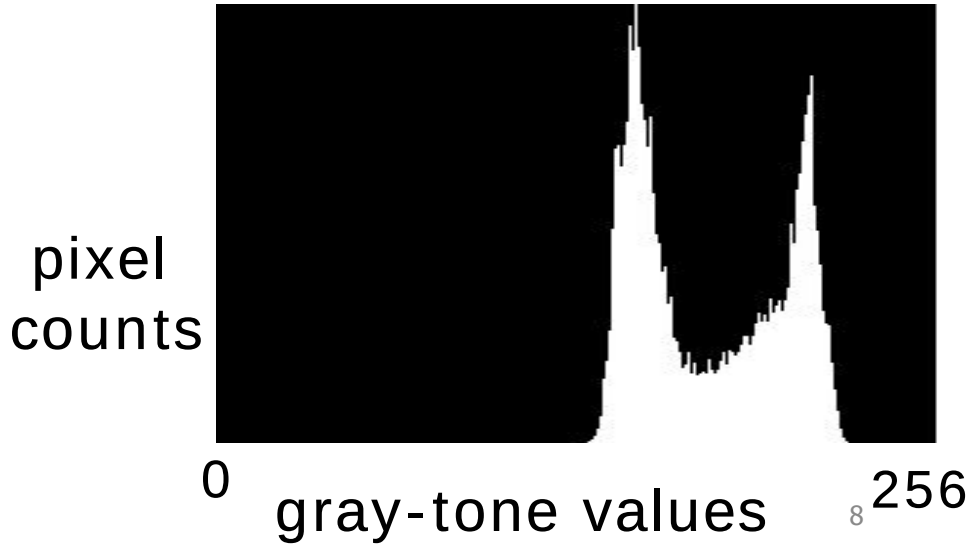
Useful Operations

1. Thresholding a gray-tone image
2. Determining good thresholds
3. Connected components analysis
4. Binary mathematical morphology
5. All sorts of feature extractors (area, centroid, circularity, …)

Thresholding

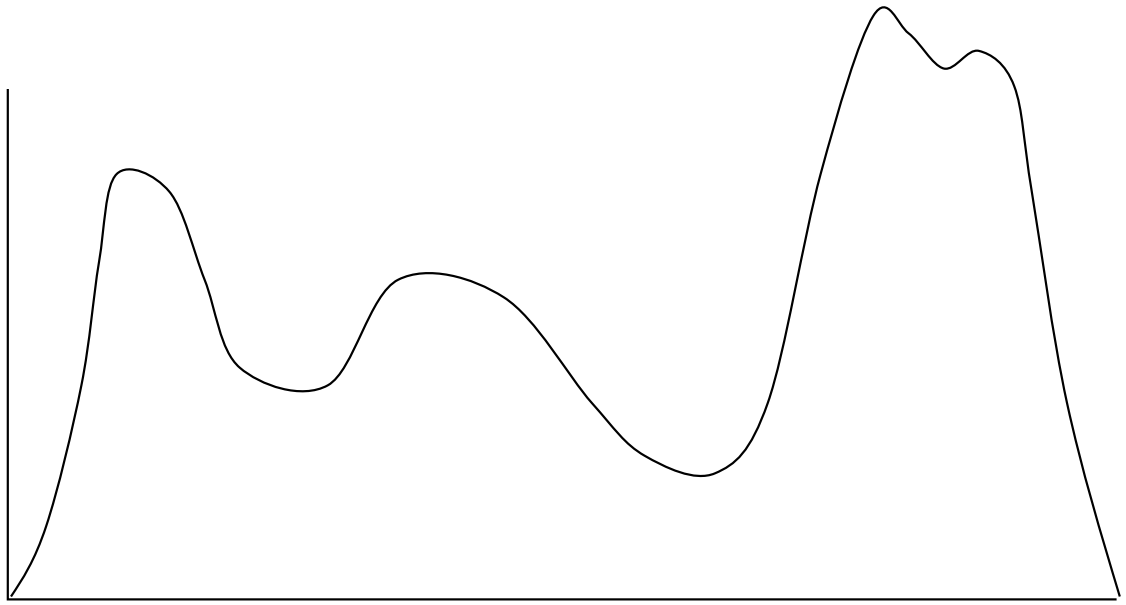
- Background is black
- Healthy cherry is bright
- Bruise is medium dark
- Histogram shows two cherry regions (black background has been removed)





Histogram-Directed Thresholding

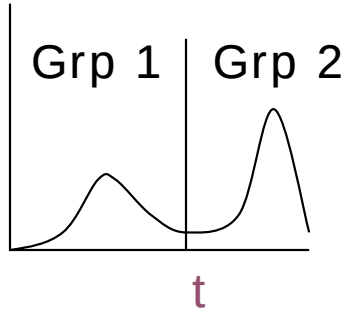
How can we use a histogram to separate an image into 2 (or several) different regions?



Is there a single clear threshold? 2? 3?

Automatic Thresholding: Otsu's Method

Assumption: the histogram is bimodal



Method: find the threshold t that minimizes the weighted sum of within-group variances for the two groups that result from separating the gray tones at value t.

Thresholding Example



original gray tone image



binary thresholded image

Auto Thresholding

- Auto threshold implements the iterative selection method.
- This method works by comparing the number of target or foreground pixel values (T1) with the number of background or background pixel values (T2).

$$T\,0 = (T\,1 + T\,2) / 2$$

dimana :

T0 = *threshold value*
T1 = average target/objek/*foreground*
T2 = average *background*

contoh Auto Thresholding :

$$\begin{pmatrix} 255 & 255 & 255 & 198 & 198 \\ 198 & 255 & 0 & 198 & 255 \\ 255 & 0 & 0 & 0 & 198 \\ 198 & 255 & 0 & 198 & 255 \\ 255 & 255 & 255 & 198 & 198 \end{pmatrix}$$

- Initial initialization:
  The number of pixels with a value of 255 = 11
  The number of pixels with a value of 198 = 9
  The number of pixels with a value of 0 is = 5
  Average pixel value of T0 = ((255*11) + (198*9) + 0) / 25
  = 183.48 ~ 184

Auto Thresholding

***Looping* 1:**
Average pixel *foreground* (T1)
= (0*5) / 5 = 0
Average pixel *background* (T2)
= ((198*9) + (255*11)) / 20 = 229.35 ~ 230
If T0 = (T1+T2) / 2 ? No -> (T1+T2) / 2 = 115

***Looping* 2:**
Average value pixel (T0)
= (T1+T2) / 2 = 115
Average pixel *foreground* (T1)
= (0*5) / 5 = 0
Average pixel *background* (T2)
= ((198*9) + (255*11)) / 20 = 230
If T0 = (T1+T2) / 2 ? true, (T1+T2) / 2 = 115
STOP

Auto Thresholding contoh :

- the optimal threshold value is 115
- Binary image values are as follows:

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Connected Components Labeling

Once you have a binary image, you can identify and then analyze each **connected set of pixels**.

The connected components operation takes in a binary image and produces a **labeled image** in which each pixel has the integer label of either the background (0) or a component.



binary image after morphology



connected components

Methods for CC Analysis

1. Recursive Tracking (almost never used)
2. Parallel Growing (needs parallel hardware)
3. Row-by-Row (most common)

- Classical Algorithm (see text)
- Efficient Run-Length Algorithm (developed for speed in real industrial applications)

Run-length encoding

- Use the lengths of the runs of 1 pixels to represent an image.
  - run : a set of adjacent pixels that lie on the same row of an image

1	1	1	0	0	0	1	1	0	0	0	1	1	1	1	0	1	1	0	1	1	1
0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1
1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1

Start position & lengths of runs of 1's for each row

(1,3) (7,2) (12,4) (17,2) (20,3)

(5,13) (19,4)

(1,3) (17,6)

Equivalent Labels

Original Binary Image

0 0 0 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 1
0 0 0 1 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1
0 0 0 1 1 1 1 1 0 0 1 1 1 1 0 0 1 1 1
0 0 0 1 1 1 1 1 1 0 1 1 1 1 0 0 1 1 1
0 0 0 1 1 1 1 1 1 **1** 1 1 1 1 0 0 1 1 1
0 0 0 1 1 1 1 1 1 1 1 1 1 1 0 0 1 1 1
0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0 0 0 1 1 1 1 1 1 0 0 0 0 1 1 1 1 1

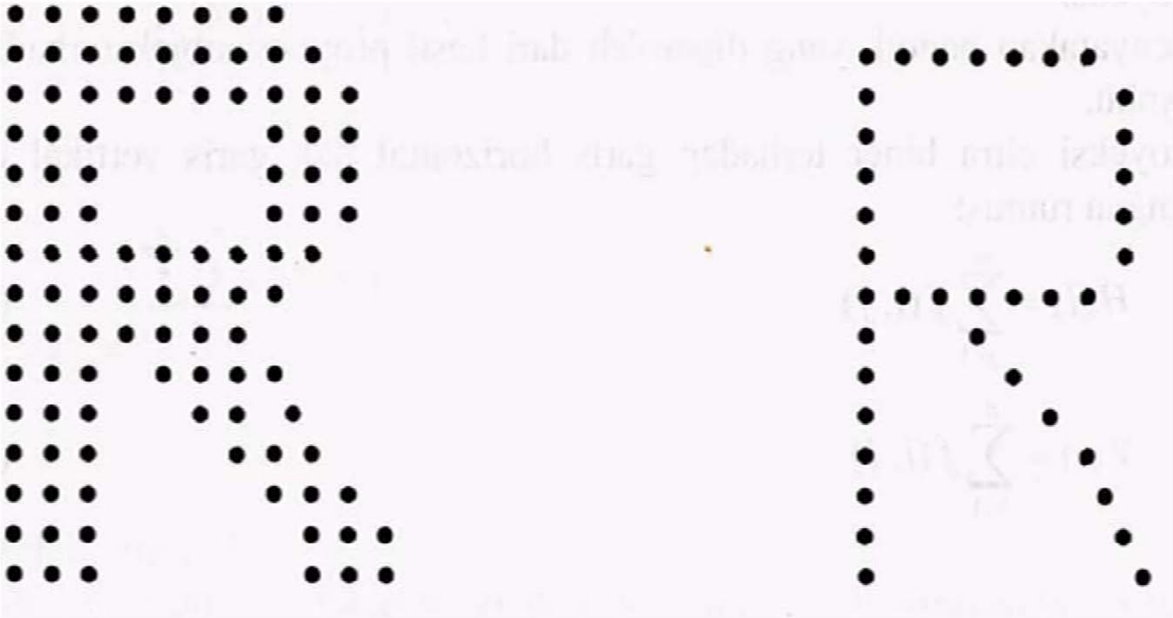
Equivalent Labels

The Labeling Process

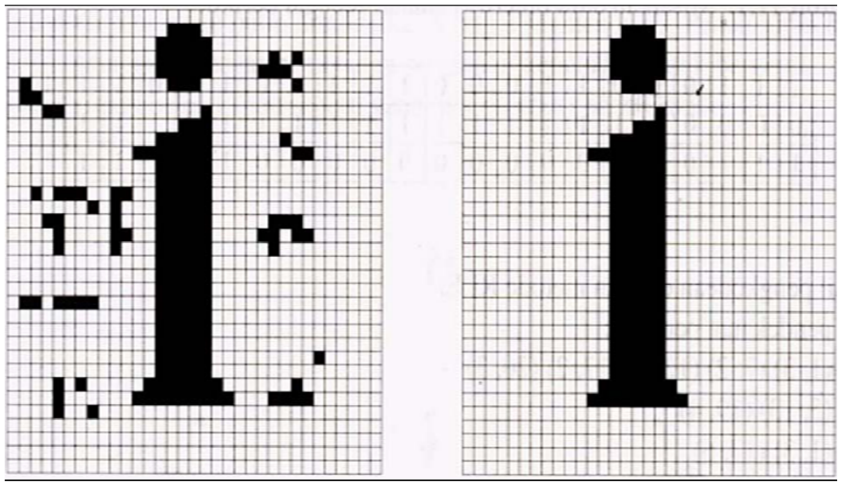
0 0 0 1 1 1 0 0 0 0 2 2 2 2 0 0 0 3
0 0 0 1 1 1 1 0 0 0 2 2 2 2 0 0 3 3
0 0 0 1 1 1 1 1 0 0 2 2 2 2 0 0 3 3 3
0 0 0 1 1 1 1 1 1 0 2 2 2 2 0 0 3 3 3
0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 0 0 3 3 3
0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 0 0 3 3 3
0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0 0 0 1 1 1 1 1 1 0 0 0 0 1 1 1 1 1

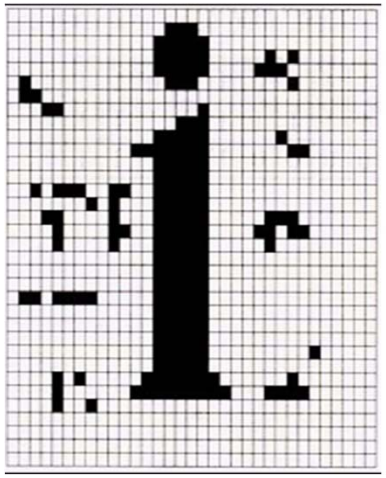
$1 \equiv 2$
$1 \equiv 3$

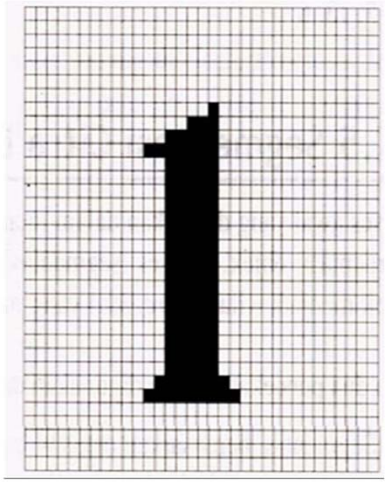
Morphological Thinning



Size Filtering







T=10

T=25